

Aperture — Low-Level Design

Module: Streaming

Version: v1.0

Module Owner: Availability Platform

Dependencies:

- Metadata Module
 - Users Module
 - Discovery Module
 - External Streaming Provider Data (via TMDb)
-

1. Overview

The Streaming module manages where users can watch movies and TV shows.

Rather than embedding streaming availability directly into metadata, Aperture treats streaming information as a dynamic domain with its own synchronization lifecycle.

This separation reflects the fact that streaming availability changes far more frequently than movie metadata.

The module enriches content rather than owning it.

2. Design Goals

The Streaming module should:

- Display current streaming availability
 - Support multiple regions
 - Differentiate subscription, rental, and purchase options
 - Update independently from metadata
 - Integrate naturally with discovery and recommendations
 - Degrade gracefully when provider data is unavailable
-

3. Module Ownership

Owns

- Streaming provider mappings
 - Regional availability
 - Availability synchronization
 - Provider metadata
 - Watch options
-

Reads

- Movies
 - TV Shows
 - User region (or selected region)
-

Never Modifies

- Movie metadata
 - TV metadata
 - Reviews
 - Lists
 - User profiles
-

4. Responsibilities

The Streaming module owns:

Providers

Examples:

- Netflix
 - Prime Video
 - Disney+
 - Apple TV
 - Hulu
 - Max
-

Availability

Track:

- Subscription
 - Rental
 - Purchase
 - Free with ads (future)
-

Regional Data

Availability differs by country.

The module should always resolve streaming options relative to a user's selected region.

Synchronization

- Initial import
 - Scheduled refresh
 - Provider updates
-

5. High-Level Workflow

Streaming Request



Resolve User Region



Lookup Availability



Availability Cached?



Yes → Return Data



No



Fetch Latest Provider Data



Normalize



Cache



Return Response

6. Data Flow

User



Movie Page



Streaming Service



Region Resolver



Availability Repository



Cache



Database



External Provider Refresh (if needed)



Response

7. Data Model

Primary entities:

Streaming Provider

Stores:

- name
 - logo
 - provider ID
 - provider type
-

Availability

Stores:

- movie / TV reference
 - provider
 - country
 - watch type
 - last updated
-

Future entities:

- Provider packages
 - Subscription tiers
 - Availability history
-

8. Synchronization Strategy

Streaming availability changes frequently.

Updates should occur independently of metadata synchronization.

Three mechanisms:

On-Demand

Refresh stale records when requested.

Scheduled

Refresh high-traffic titles periodically.

Background Jobs

Incrementally update provider availability.

Avoid refreshing the entire catalog unnecessarily.

9. Region Resolution

Streaming results depend on geography.

Region selection order:

1. User preference
2. Browser locale (optional)
3. Default application region

Users should be able to override the detected region.

10. Business Rules

Availability is always scoped by:

- title

- provider
- country
- watch type

Expired availability should not be deleted immediately; it should be marked stale until refreshed to reduce transient inconsistencies.

11. Discovery Integration

Streaming availability influences discovery.

Examples:

- Available on your subscriptions
- New on Netflix
- Leaving soon (future)
- Included with Prime

The Discovery module consumes these signals but does not own them.

12. Recommendation Integration

Streaming availability may affect recommendation ranking.

Examples:

- Boost titles available on a user's preferred platforms
- Surface highly rated content the user can watch immediately

Availability should influence ranking, not determine it.

13. Caching Strategy

Cache candidates:

- provider lists
- regional availability
- popular title availability

Short cache lifetimes are preferred because licensing changes regularly.

14. Performance Considerations

Potential bottlenecks:

- High-traffic regions
- Frequent availability updates
- Multiple provider lookups

Mitigations:

- Redis caching
 - Background refresh
 - Indexed lookups by region and title
 - Batched synchronization jobs
-

15. Security Considerations

Provider synchronization endpoints should not be publicly accessible.

External provider credentials must remain server-side.

User region preferences should be treated as profile settings and not exposed unnecessarily.

16. Error Handling

Representative failures:

- Provider unavailable
- Region unsupported
- Synchronization timeout
- Missing provider mapping

Fallback behavior:

- Serve cached availability
- Indicate that availability information is temporarily unavailable
- Continue displaying the rest of the movie page

Streaming failures should never prevent users from accessing movie metadata.

17. Testing Strategy

Unit Tests

- Region resolution
 - Provider mapping
 - Availability normalization
-

Integration Tests

- Metadata integration
 - Synchronization pipeline
 - Cache behavior
-

API Tests

- Availability lookup
 - Region switching
 - Provider filtering
-

Performance Tests

Measure:

- Lookup latency
 - Cache hit ratio
 - Synchronization throughput
-

18. Future Service Extraction

The Streaming module is a moderate candidate for extraction.

Reasons:

- Independent update frequency
- External integrations
- Region-specific workloads

Potential published events:

- AvailabilityUpdated

- ProviderAdded
- ProviderRemoved

Potential consumers:

- Discovery
- Recommendation Engine
- Notifications
- Analytics

Service extraction should only occur if streaming synchronization becomes operationally significant.

19. Future Evolution

Potential enhancements:

- User subscription management
- Platform-specific notifications
- Leaving-soon alerts
- Price tracking
- Availability history
- Deep links to providers
- Provider reliability scoring

The module should support these capabilities without altering existing APIs.

20. Interview Discussion Points

Be prepared to explain:

- Why separate Streaming from Metadata?
 - Why synchronize availability independently?
 - How do regional differences affect the data model?
 - How would you reduce API calls to external providers?
 - How should stale availability be handled?
 - When would Streaming become its own service?
-

21. Summary

The Streaming module enriches Aperture's content platform with dynamic, region-aware availability information.

By treating streaming as an independent domain, the architecture accommodates frequent licensing changes, simplifies synchronization, and enables discovery experiences that help users immediately answer the question: "Where can I watch this?"